EventHub
CONNECT. PARTICIPATE. EXPERIENCE.

PROJETO ACADÊMICO | UECE

Relatório Final do Projeto EventHub

Plataforma de gerenciamento de eventos acadêmicos e profissionais

ARQUITETURA DE MICROSSERVIÇOS

Comunicação síncrona por APIs HTTP, mensageria assíncrona, persistência por domínio e execução com Docker Compose.

NOTA DE ESCOPO
O projeto está concluído para o escopo acadêmico atual. Esta edição apresenta os oito serviços de domínio, o provedor de pagamento simulado e os fluxos integrados da plataforma, com foco nas responsabilidades, nos dados e na colaboração entre os microsserviços.

Universidade Estadual do Ceará (UECE)

12 de novembro de 2026

Relatório técnico baseado na implementação disponível no repositório.

Sumário

1 Visão geral do projeto 2

2 Problema abordado 2

3 Objetivos 3

4 Público-alvo e perfis 3

5 Requisitos funcionais 4

6 Requisitos não funcionais 5

7 Decisões arquiteturais e evolução 5

8 Tecnologias utilizadas 10

9 Organização do repositório 11

10 Arquitetura da solução 12

11 Módulos e microsserviços 14

12 Funcionalidades implementadas 18

13 Regras de negócio e controles 19

14 Dados e persistência 21

15 Fluxos de negócio distribuídos 23

16 Integrações e comunicação 25

17 Ambiente e execução 26

18 Validação e rastreabilidade 27

19 Uso de inteligência artificial na documentação 27

20 Estado atual da entrega 28

21 Limites do escopo atual 28

22 Planejamento e resultado alcançado 29

23 Encerramento 29

Referências técnicas 31

1. Visão geral do projeto

O EventHub reúne os processos de um evento em uma plataforma web: organização da programação, reserva de vagas, acompanhamento de inscrições, pagamento simulado, comunicação com participantes, registro de presença e emissão de certificados. A versão apresentada está concluída no escopo atual do projeto acadêmico.

O resultado entregue compreende uma interface React e um backend distribuído em serviços Node.js e Python, executados com Docker Compose. A unidade de participação é a atividade de um evento: é nela que se definem capacidade, preço, inscrição, presença e certificação. O relatório enfatiza como essas responsabilidades foram separadas e como os serviços cooperam para executar os fluxos completos.

Dimensão	Resultado implementado
Domínios	Identity, Events, Registrations, Payments, Notifications, Attendance, Certificates e Reporting.
Integração adicional	Payment Provider, simulador separado, com API e persistência próprias.
Experiência web	Acesso por perfil, catálogo, gestão de eventos, inscrições e dashboards.
Operação local	Compose, proxy reverso, bancos por domínio, mensageria, healthchecks e scripts operacionais.
Limite financeiro	Pagamentos e receitas demonstrativos, sem cobrança ou liquidação real.

As escolhas de simulação financeira e execução local delimitam a versão entregue. Elas não são tratadas como funcionalidades pendentes desta etapa.

Evidência: `compose.yaml`; `frontend/src/App.tsx`; `services/identity/src/users/service.js`; `services/events/app/domain.py`.

2. Problema abordado

A gestão de eventos reúne processos com regras diferentes: divulgação da programação, limite de vagas, reservas pagas, lista de espera, comunicação, credenciamento e certificação. Quando esses processos são tratados de forma isolada, o organizador precisa conciliar informações e o participante perde uma visão consistente de sua participação.

O problema técnico central é coordenar esses processos sem concentrar todas as responsabilidades em um único serviço ou compartilhar diretamente suas tabelas. Uma inscrição precisa respeitar a capacidade da atividade; um pagamento aprovado deve corresponder a uma reserva válida; um certificado depende da presença e do término da atividade.

O EventHub resolve esse recorte com serviços de domínio que mantêm suas regras e seus dados, colaborando por contratos explícitos. As decisões que exigem resposta imediata utilizam HTTP; notificações e atualização de projeções utilizam mensagens e mecanismos de recuperação.

3. Objetivos

3.1 Objetivo geral alcançado

Entregar uma plataforma web integrada para gestão de eventos acadêmicos e profissionais, aplicando uma arquitetura de microsserviços aos fluxos de participação e organização.

3.2 Objetivos específicos atendidos

- Separar identidade, programação, inscrições, pagamentos, comunicação, presença, certificados e relatórios em serviços com responsabilidades próprias.
- Preservar a propriedade dos dados de cada domínio e integrar consultas e comandos por APIs.
- Controlar capacidade, reservas e lista de espera por atividade, com transações locais e idempotência.
- Coordenar uma fronteira externa de pagamento simulado e recuperar resultados pendentes por reconciliação.
- Desacoplar notificações e projeções das operações principais por outbox e mensageria.
- Disponibilizar presença, certificação e indicadores na experiência web dos perfis de uso.
- Reunir aplicação e infraestrutura em uma composição local documentada e reproduzível.

4. Público-alvo e perfis

O **participante** consulta eventos, inscreve-se em atividades, acompanha reservas e pagamentos simulados, apresenta sua credencial e acessa seus certificados. O **organizador** é um usuário individual vinculado a uma instituição e responsável pelos eventos que administra. O vínculo institucional e a propriedade do evento são dados distintos: `organizationId` identifica a instituição, enquanto `ownerUserId` identifica o responsável pelo evento.

Identity mantém as contas e os vínculos institucionais; Events utiliza esses dados para autorizar a gestão de cada evento. A existência de membros na mesma instituição não concede, por si só, a propriedade dos eventos de outro organizador.

Perfil	Atuação na plataforma
Participante	Consulta o catálogo, inscreve-se nas atividades e acompanha pagamento simulado, credencial e certificados.
Organizador	Administra seus eventos, programação, inscrições, presença, modelos de certificado e indicadores.
Instituição	Representa o vínculo organizacional dos organizadores; sua identidade não substitui a conta individual nem a propriedade dos eventos.

Evidência: `services/identity/src/users/service.js`; `services/events/app/domain.py`; `frontend/src/App.tsx`.

5. Requisitos funcionais

Os requisitos abaixo sintetizam as capacidades implementadas na versão entregue e sua distribuição entre os serviços.

Código	Requisito implementado	Responsabilidade
RF01	Cadastrar e autenticar usuários, manter perfis e revogar sessões.	Identity.
RF02	Manter instituições e vínculos dos organizadores, distinguindo vínculo e responsabilidade pelo evento.	Identity e Events.
RF03	Criar, editar, publicar e cancelar eventos e suas atividades.	Events.
RF04	Inscrever participantes por atividade, controlar vagas, reservas e lista de espera.	Registrations.
RF05	Executar pagamentos simulados e reconciliar os resultados com a inscrição.	Payments, Payment Provider e Registrations.
RF06	Comunicar mudanças de inscrição, resultados financeiros simulados e avisos de evento.	Notifications; os fatos são produzidos pelos domínios responsáveis.
RF07	Registrar presença por inscrição, com check-in manual ou credencial QR.	Attendance.
RF08	Emitir e verificar certificados por atividade concluída, conforme inscrição e presença.	Certificates.
RF09	Consolidar inscrições, ocupação, presença e valores simulados.	Reporting.
RF10	Restringir a gestão ao organizador responsável e validar o acesso aos recursos.	Identity e serviços de domínio.

6. Requisitos não funcionais

Código	Requisito	Mecanismo implementado
RNF01	Separação de responsabilidades e dados	Oito domínios e um provedor utilizam nove bancos lógicos, sem consultas SQL de negócio entre serviços.
RNF02	Autenticação e sessão	JWT assinado e verificação de revogação confirmam a validade atual da sessão.
RNF03	Autorização por recurso	Vínculo institucional e `ownerUserId` sustentam a validação do organizador responsável.
RNF04	Integridade do inventário	Transações locais, bloqueios SQL e restrições preservam a capacidade das atividades.
RNF05	Tratamento de repetições	Chaves de idempotência, referências únicas e recibos persistidos reconhecem operações repetidas.
RNF06	Comunicação recuperável	Outbox e RabbitMQ preservam intenções de publicação; consumidores registram processamento e tentativas.
RNF07	Reconciliação entre serviços	Workers retomam pagamentos; Reporting consulta as fontes para atualizar suas projeções.
RNF08	Reprodutibilidade local	Docker Compose define os 14 componentes e suas dependências de infraestrutura.

Esses mecanismos descrevem propriedades implementadas. Não representam uma medição de alta disponibilidade, desempenho sob carga de produção ou entrega de mensagens exatamente uma vez.

7. Decisões arquiteturais e evolução

A arquitetura entregue organiza as regras por domínio e combina dois modos de integração. HTTP atende às decisões imediatas de autenticação, reserva, pagamento, presença e elegibilidade. A mensageria propaga efeitos que podem ser concluídos após a operação principal, como notificações e atualização de projeções.

Decisão	Aplicação na versão final
Serviços por domínio	Regras e persistência mantidas pelo serviço responsável; contratos externos passam pelo gateway.
Backend com duas tecnologias	Express nos serviços Node.js e FastAPI nos serviços Python, sem exigir uniformidade da implementação interna.
Banco próprio por serviço	Nove bancos lógicos e usuários separados em uma instância PostgreSQL local.
Transações locais e outbox	A confirmação da inscrição e a intenção de comunicação são gravadas na mesma transação.
Provedor externo simulado	Payment Provider possui contrato e banco próprios; Payments coordena a integração e a recuperação.
Atividade como unidade	Capacidade, preço, inscrição, presença e certificado são definidos por atividade.

A evolução representada neste relatório é a consolidação desses mecanismos nos fluxos completos. Attendance, Certificates e Reporting pertencem à implementação entregue. A ampliação para outra infraestrutura ou para cobrança real constitui um escopo posterior.

Os quatro registros de decisão arquitetural (*Architecture Decision Records* — ADRs) a seguir detalham o contexto, as alternativas analisadas, a decisão adotada e suas consequências na versão atual.

7.1 ADR-01: decomposição em oito serviços de domínio e granularidade por atividade

Contexto: O sistema reúne processos com regras e ritmos de evolução distintos. A abertura de inscrições pode concentrar requisições, enquanto notificações, certificados e relatórios possuem outros padrões de uso. Cada atividade de um evento mantém capacidade, preço, lista de espera e critérios de certificação próprios.

Alternativas analisadas: Um monólito modular simplificaria a implantação inicial, mas reuniria a implantação e a replicação do backend em uma unidade maior. A criação de um microsserviço por requisito funcional ampliaria o número de fronteiras e de chamadas entre regras próximas, como inscrição, inventário e lista de espera.

Decisão: Organizar a solução em oito serviços de domínio — Identity, Events, Registrations, Payments, Notifications, Attendance, Certificates e Reporting — e manter Payment Provider como serviço separado para representar a fronteira financeira simulada. A atividade é adotada como unidade central de participação.

Justificativa: A consulta e a gestão da programação permanecem em Events; inscrição, inventário e espera permanecem em Registrations. Esse agrupamento mantém próximas as regras que exigem decisão transacional conjunta e separa os processos de comunicação, presença, certificação e composição de indicadores.

Consequências: A separação favorece a evolução e a possibilidade de replicação por serviço, além de permitir que efeitos assíncronos sejam processados após a operação principal. Em contrapartida, a composição passa a reunir 14 componentes, e verificações de acesso e elegibilidade exigem chamadas entre serviços. O compartilhamento de infraestrutura e as dependências HTTP continuam relevantes para a disponibilidade do conjunto.

Evidência: `compose.yaml`; `services/events/app/domain.py`; `services/registrations/app/domain.py`.

7.2 ADR-02: controle de concorrência e idempotência no inventário de vagas

Contexto: Requisições simultâneas podem disputar as últimas vagas de uma atividade. Repetições provocadas por falhas de rede também precisam ser reconhecidas para preservar a capacidade e evitar inscrições ativas duplicadas, conforme os controles de integridade e repetição descritos em RNF04 e RNF05.

Alternativas analisadas: O controle otimista por versionamento poderia exigir novas tentativas quando várias operações disputassem o mesmo inventário. Uma fila sequencial única de pedidos de inscrição exigiria outro modelo de retorno ao participante para informar confirmação, reserva paga ou entrada na espera.

Decisão: Utilizar transações locais em `registration_db`, com bloqueio pessimista do inventário por `SELECT FOR UPDATE`, bloqueio consultivo por usuário e chaves de idempotência persistidas. Restrições e verificações locais complementam o controle de inscrições ativas duplicadas.

Justificativa: A leitura da ocupação, a manutenção das reservas e a decisão sobre uma nova inscrição são coordenadas dentro da fronteira transacional de Registrations. O bloqueio do inventário serializa as mutações da mesma atividade; o bloqueio por usuário e a referência persistida permitem reconhecer a repetição de uma solicitação.

Consequências: O mecanismo protege a capacidade da atividade e permite retomar chamadas compatíveis sem criar outra inscrição. A disputa por uma mesma atividade também pode aumentar o tempo de espera por bloqueios. A duração das transações e a ordem de aquisição dos bloqueios precisam permanecer controladas nas operações de inscrição, cancelamento, expiração e promoção da espera.

Evidência: `services/registrations/app/domain.py`; `services/registrations/app/migrations/`.

7.3 ADR-03: efeitos assíncronos com Transactional Outbox e RabbitMQ

Contexto: Mudanças nas inscrições e decisões de pagamento precisam gerar mensagens aos participantes e atualizar os indicadores, conforme RF06 e RF09. A conclusão da operação principal deve ser separada do processamento desses efeitos, permitindo recuperar falhas de comunicação e indisponibilidades temporárias dos consumidores ou do broker.

Alternativas analisadas: Chamadas HTTP síncronas de Registrations para Notifications e Reporting colocariam esses serviços na cadeia imediata de resposta da inscrição. A publicação direta no RabbitMQ após o commit deixaria um intervalo entre a alteração do banco e a publicação no qual uma falha poderia interromper a comunicação sem preservar a intenção de envio.

Decisão: Gravar o evento na outbox de `registration_db` na mesma transação local da mudança da inscrição. Um worker publica os registros pendentes na exchange durável `eventhub`, do tipo *topic*, utilizando confirmação de publicação e `mandatory`. Notifications e Reporting possuem filas e controles de processamento próprios; Notifications mantém ainda a fila `notifications.failed` para mensagens que atingem o limite de tentativas.

Justificativa: A persistência conjunta vincula a alteração do estado à intenção de comunicação. O publicador pode retomar registros pendentes quando o broker estiver disponível. Como uma publicação pode ser repetida antes da gravação de sua confirmação local, o tratamento pelos consumidores considera a entrega pelo menos uma vez.

Consequências: Notificações e atualização de projeções podem ocorrer após a confirmação operacional da inscrição. Isso reduz a dependência imediata desses efeitos e introduz consistência eventual, acúmulo possível de mensagens e necessidade de acompanhar falhas. Os

consumidores mantêm controles de repetição, como `deliveries` e `processed_messages`; a entrega final por SMTP permanece sujeita às limitações descritas na seção de comunicação.

Evidência: `services/registrations/app/domain.py`; `services/notifications/app/consumer.py`; `services/reporting/app/consumer.py`.

7.4 ADR-04: integração híbrida e reconciliação de pagamentos

Contexto: Autenticação, reserva, check-in e emissão de certificados dependem da validação do estado atual das fontes. A integração com o provedor de pagamento simulado pode sofrer falhas ou atrasos, inclusive com uma aprovação recebida após o vencimento da reserva. Esses fluxos precisam combinar resposta imediata e recuperação posterior.

Alternativas analisadas: Uma coordenação inteiramente orientada a eventos exigiria outro modelo para correlacionar pedidos e apresentar resultados imediatos ao frontend. Transações distribuídas com commit em duas fases exigiriam um protocolo de coordenação e suporte transacional dos participantes, inclusive do provedor. Essa alternativa ampliaria o acoplamento operacional entre as fronteiras de serviço.

Decisão: Utilizar HTTP/JSON para validações imediatas e coordenação do pagamento, mantendo a transação intermediária `PENDING` persistida em Payments e um worker para sua reconciliação. Registrations permanece como autoridade sobre a confirmação da vaga e a validade da reserva. A mensageria atende aos efeitos posteriores; Reporting reconcilia suas projeções pelo consumidor e durante as consultas.

Justificativa: Contratos HTTP explícitos permitem a comunicação entre Node.js e Python. A persistência das decisões, as referências idempotentes e a repetição controlada permitem recuperar etapas interrompidas sem um commit global. A confirmação em Registrations preserva a validade atual da reserva quando o resultado externo chega com atraso.

Consequências: A solução permite recuperar pagamentos pendentes e mantém a decisão de ocupação no serviço responsável pelo inventário. As cadeias síncronas preservam acoplamento temporal: o gateway depende de Identity para verificar a sessão, e Certificates consulta diferentes serviços para avaliar a emissão. A latência dessas dependências pode se acumular, enquanto falhas intermediárias podem manter o resultado pendente até a reconciliação.

Evidência: `services/payments/src/payments.js`; `services/payments/src/provider.js`; `services/payment-provider/src/app.js`; `services/registrations/app/domain.py`; `services/reporting/app/domain.py`; `gateway/src/routes/business.js`.

7.5 O que ganhamos e o que pagamos por distribuir o sistema?

Ganhamos a possibilidade de escalar serviços seletivamente, isolamento parcial de falhas e independência de stacks. Pagamos com consistência eventual nas projeções e entregas assíncronas, mais etapas de espera nas chamadas HTTP e maior esforço operacional para coordenar contêineres, filas, outbox e recuperação de falhas.

Ganhos

- **Escalabilidade seletiva:** a separação permite planejar a replicação de Registrations conforme a demanda de inscrições, sem exigir a mesma replicação de certificados, notificações e todos os demais serviços.

- **Isolamento de falhas:** falhas no processamento de e-mails ou na atualização assíncrona de relatórios podem ser tratadas após a confirmação da inscrição. A operação principal pode ser concluída enquanto esses efeitos aguardam recuperação.
- **Stacks independentes:** serviços como Identity e Payments utilizam Node.js e Express, enquanto Events e Registrations utilizam Python e FastAPI. Cada implementação pode evoluir suas dependências e organização interna, preservando os contratos entre os domínios.

Custos

- **Consistência eventual:** uma inscrição pode estar confirmada antes de o e-mail ser entregue ou de a projeção de Reporting ser atualizada. Os consumidores precisam acompanhar o estado das fontes e tratar mensagens repetidas ou atrasadas.
- **Latência e dependências em cadeia:** Certificates consulta quatro serviços — Identity, Events, Registrations e Attendance — para autorizar e validar a emissão. As chamadas HTTP acrescentam comunicação de rede e etapas de espera; a indisponibilidade de uma fonte pode impedir a conclusão imediata. O gateway também consulta Identity para verificar a validade atual da sessão.
- **Sobrecarga operacional:** a execução exige coordenar 14 componentes, suas imagens, configurações, migrações, credenciais e verificações de prontidão. A outbox e a mensageria acrescentam publicação com confirmação, retentativas, deduplicação, acompanhamento de filas e investigação por logs correlacionados.

Esses ganhos expressam possibilidades e mecanismos da arquitetura adotada. A composição local mantém infraestrutura compartilhada e dependências síncronas; a escalabilidade seletiva e o isolamento de falhas não representam resultados de desempenho ou disponibilidade medidos nesta entrega.

Evidência: `compose.yaml`; `services/certificates/app/domain.py`; `services/registrations/app/domain.py`; `services/notifications/app/consumer.py`; `services/reporting/app/consumer.py`; `gateway/src/security/sessions.js`.

8. Tecnologias utilizadas

Camada	Tecnologia	Uso
Interface	React, TypeScript e Vite	Rotas, autenticação, catálogo e painéis web.
Entrada HTTP	Nginx e Express	Proxy reverso, gateway e encaminhamento das APIs.
Backend Node.js	Node.js 24 e Express	Identity, Payments e Payment Provider.
Backend Python	Python 3.13 e FastAPI	Events, Registrations, Notifications, Attendance, Certificates e Reporting.
Persistência	PostgreSQL 17	Nove bancos lógicos com usuários próprios.
Mensageria	RabbitMQ 4 e Pika	Exchange, filas e consumidores assíncronos.
Documentos	ReportLab e Pillow	Geração e composição de certificados PDF.
Execução	Docker e Docker Compose	Imagens e composição local dos 14 componentes.

Node.js 24 e Python 3.13 materializam as duas tecnologias de backend da solução. As aplicações Node utilizam Express, `pg` e, na identidade e no gateway, `jose`. As aplicações Python utilizam FastAPI, Uvicorn e `psycopg`; consumidores RabbitMQ utilizam Pika. Certificates utiliza ReportLab e Pillow para os documentos.

Evidência: `compose.yaml`; Dockerfiles, `package.json` e `requirements.txt` dos respectivos componentes.

9. Organização do repositório

A estrutura separa a interface, o ponto de entrada das APIs, as implementações de domínio e os recursos operacionais. Cada diretório de serviço mantém seu Dockerfile e suas dependências.

```
eventhub/
  frontend/           # React, TypeScript e Vite
  gateway/            # API Gateway em Express
  services/
    identity/         # Contas, sessoes e instituicoes
    events/           # Eventos e atividades
    registrations/    # Inscricoes, inventario e outbox
    payments/         # Pagamentos e reconciliacao
    payment-provider/ # Integracao financeira simulada
    notifications/    # Consumidor e entrega de mensagens
    attendance/       # Presenca e credenciamento
    certificates/     # Emissao e verificacao de PDFs
    reporting/        # Projecoes, indicadores e CSV
  infra/
    db/               # Provisionamento dos bancos
    nginx/            # Proxy reverso
  scripts/            # Configuracao, auditoria e rastreamento
  docs/               # Fonte deste relatorio e figuras
  compose.yaml        # Composicao da aplicacao
  .env.example        # Exemplo de configuracao
  README.md           # Preparacao e operacao
```

As aplicações Node.js organizam sua implementação em `src/`; as aplicações Python utilizam `app/`. As migrações e os mecanismos de inicialização permanecem junto dos serviços que possuem os dados.

10. Arquitetura da solução

O sistema utiliza uma arquitetura de microsserviços conteinerizada. Cada serviço de domínio possui implementação, dependências, imagem e persistência próprias. As relações entre domínios são resolvidas por APIs e, nos fluxos de propagação de eventos, por mensageria. Não há consultas SQL de negócio que atravessem os bancos dos serviços.

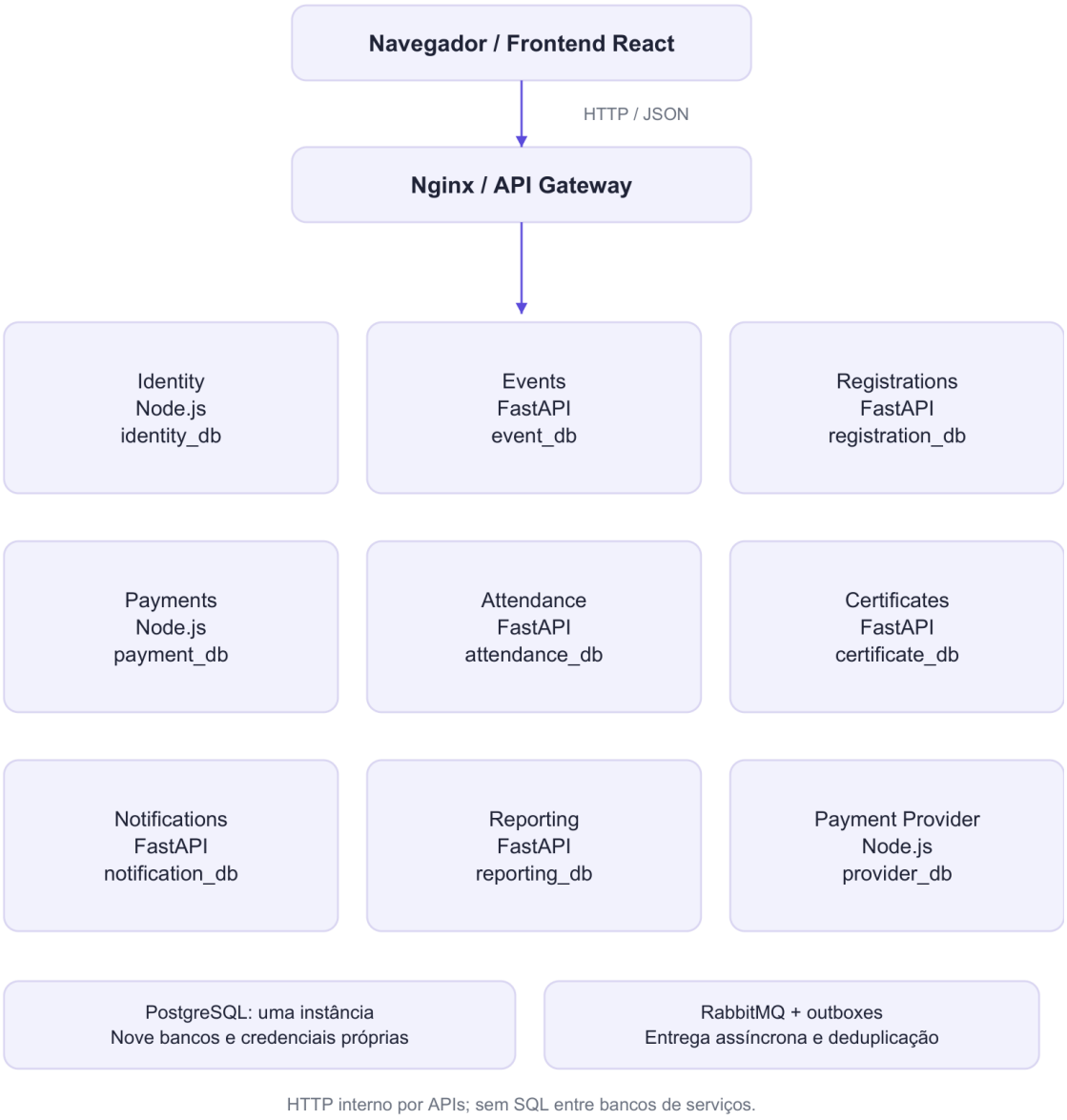


Figura 1: Visão geral da arquitetura: oito serviços de domínio, provedor simulado, bancos lógicos e mensageria. O proxy Nginx e o API Gateway são componentes distintos da entrada HTTP.

10.1 Entrada HTTP e componentes de apoio

O Nginx é o ponto de entrada publicado em `127.0.0.1:8080`, na configuração padrão. As rotas da interface são encaminhadas ao frontend e o prefixo `/api/` ao gateway. O frontend é compilado pelo Vite e servido por outro processo Nginx na imagem da aplicação web.

O gateway, desenvolvido em Express, concentra os contratos externos e encaminha as operações aos respectivos serviços. Também propaga identificadores de requisição e correlação, aplica limites de corpo e preserva respostas PDF e CSV. As rotas internas de confirmação de pagamento, consulta entre serviços e reconciliação não são publicadas como rotas externas do gateway.

Os 14 componentes do Compose correspondem a oito domínios, um provedor simulado, frontend, gateway, proxy Nginx, PostgreSQL e RabbitMQ. Esse total não deve ser interpretado como 14 microsserviços de negócio.

Evidência: `infra/nginx/nginx.conf`; `gateway/src/app.js`; `gateway/src/routes/business.js`; `frontend/Dockerfile`.

10.2 Organização da interface

O frontend mantém as rotas e os clientes HTTP de acesso ao gateway. O contexto de autenticação e as restrições de navegação organizam a experiência por perfil; as decisões de autorização e de negócio são aplicadas novamente no backend.

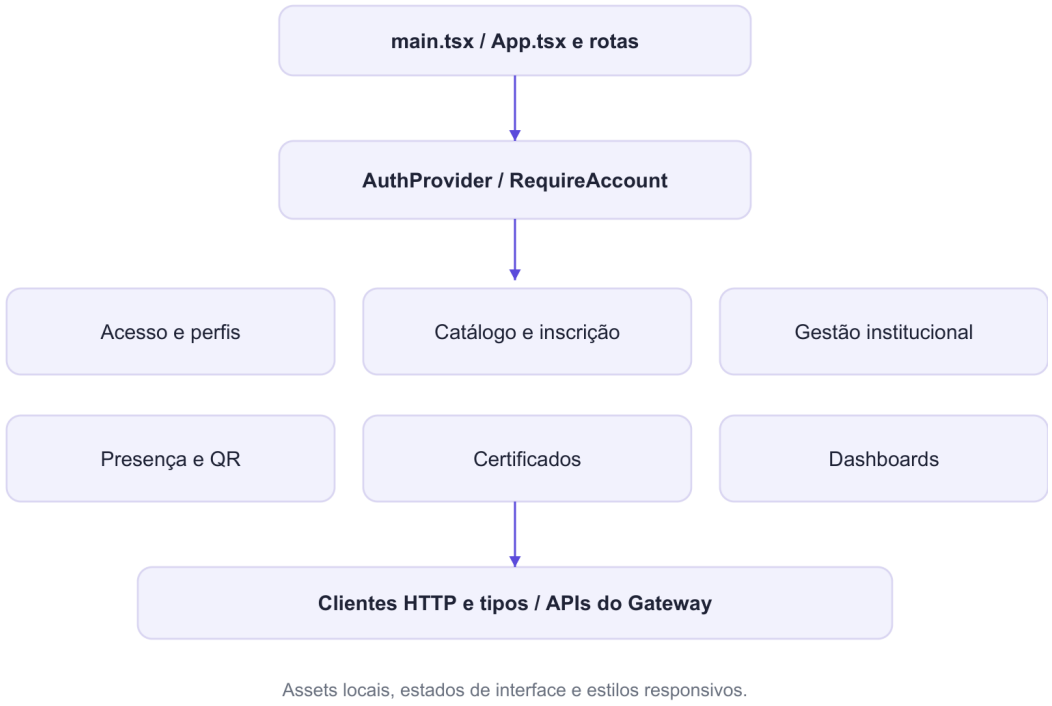


Figura 2: Organização dos módulos da interface e sua integração com os contratos HTTP.

11. Módulos e microsserviços

11.1 Frontend e API Gateway

O frontend fornece a experiência de acesso, catálogo, gestão e acompanhamento. O gateway apresenta os contratos HTTP externos, verifica a sessão e encaminha as chamadas. Nenhum dos dois assume a propriedade do inventário, dos pagamentos ou dos demais dados de negócio.

As subseções seguintes descrevem os oito microsserviços de domínio e, separadamente, o Payment Provider. Cada descrição relaciona responsabilidade, tecnologia, dados e contratos representativos.

11.2 Identity: identidade e instituições

Tecnologia: Node.js 24, Express, `pg` e `jose`.

Dados próprios: `identity_db`.

Identity centraliza cadastro, autenticação, perfil, instituições e membros. O cadastro distingue os perfis `PARTICIPANT` e `ORGANIZER`, e a autorização preserva o vínculo institucional único do organizador. As senhas utilizam scrypt; os tokens JWT são assinados com RSA/RS256 e possuem duração de uma hora.

O serviço mantém mecanismos de revogação de sessão, versão de credenciais e recuperação de senha. O gateway verifica a assinatura do JWT e consulta Identity para confirmar que a sessão continua válida. A saída da conta e as alterações de credenciais, portanto, não dependem apenas de aguardar o token expirar. As mensagens de recuperação e confirmação de e-mail utilizam uma outbox própria de Identity, distinta da outbox de eventos de inscrição.

Contratos representativos: `/auth/register`, `/auth/login`, `/users/me`, `/organizations` e `/internal/auth/verify`.

Evidência: `services/identity/src/users/`; `services/identity/src/security/`; `services/identity/src/mail/`; `gateway/src/security/sessions.js`.

11.3 Events: eventos e programação

Tecnologia: Python 3.13, FastAPI e `psycopg`.

Dados próprios: `event_db`.

Events é a fonte dos dados de eventos, atividades, palestrantes, modalidade, capacidade, períodos e preços. A gestão exige compatibilidade entre instituição, perfil e responsável pelo evento. A consulta gerenciada verifica `ownerUserId`, além do vínculo institucional atual.

O ciclo de vida utiliza os estados `DRAFT`, `PUBLISHED` e `CANCELLED`. A edição estrutural ocorre em rascunho; após a publicação, informações públicas específicas podem ser atualizadas por um contrato próprio, sem alterar silenciosamente o inventário e os preços usados nas inscrições. Alterações e cancelamentos são registrados para encaminhamento posterior a Registrations, que identifica as inscrições afetadas e produz as notificações correspondentes.

Contratos representativos: `/events`, `/events/:id`, publicação, cancelamento e atualização de informações públicas.

Evidência: `services/events/app/domain.py`; `services/events/app/announcements.py`.

11.4 Registrations: inscrições e inventário

Tecnologia: Python 3.13, FastAPI, `psycopg` e Pika.

Dados próprios: `registration_db`.

Registrations concentra a decisão sobre ocupação de vagas. O inventário de cada atividade é bloqueado com `SELECT FOR UPDATE`, e as mutações seguem uma ordem comum de bloqueio. Um bloqueio consultivo por usuário serializa a verificação das chaves de idempotência. Restrições e consultas locais impedem a criação de inscrições ativas duplicadas.

O serviço valida evento publicado, período de inscrição, atividade e opção de preço. Uma atividade gratuita confirma a inscrição; uma atividade paga cria uma reserva temporária; a lotação pode direcionar o participante à lista de espera. O preço e os dados necessários da atividade são preservados em snapshot. O prazo padrão de reserva é de 900 segundos, configurável por ambiente.

Workers tratam expiração, promoção da espera e publicação da outbox. A espera segue a ordem de criação, com o identificador como desempate. O serviço também recebe decisões internas de pagamento e mantém os respectivos recibos, além de gerar a credencial de check-in vinculada à inscrição, ao usuário e à atividade.

Contratos representativos: `/registrations`, cancelamento, disponibilidade por evento e `/internal/registrations/:id/payment`.

Evidência: `services/registrations/app/domain.py`; `services/registrations/app/migrations/`.

11.5 Payments: coordenação do pagamento simulado

Tecnologia: Node.js 24, Express e `pg`.

Dados próprios: `payment_db`.

Payments mantém a transação associada à inscrição, o valor, a decisão solicitada na simulação, a referência do provedor e o resultado reconciliado. Antes de iniciar o processo, consulta Registrations e verifica titularidade, valor positivo e reserva disponível para pagamento.

A transação é persistida como `PENDING` antes da conclusão do fluxo externo. Payments chama Payment Provider por HTTP e valida referência, valor, moeda, prazo e resultado. Em seguida, informa Registrations por um contrato interno. A confirmação efetiva da vaga permanece uma decisão de Registrations: uma resposta de aprovação do provedor não autoriza recuperar uma reserva já expirada.

Um worker consulta pagamentos pendentes e repete a reconciliação. As decisões persistidas permitem retomar o processo após falhas de comunicação, sem depender da permanência da requisição original no navegador.

Contratos representativos: `/payments/:id` e `/internal/payments/query`.

Evidência: `services/payments/src/payments.js`; `services/payments/src/provider.js`.

11.6 Payment Provider: fronteira de integração simulada

Tecnologia: Node.js 24, Express e `pg`.

Dados próprios: `provider_db`.

O provedor é separado de Payments e mantém sua própria API e suas transações. A referência da inscrição é única e funciona como limite de idempotência. Repetir uma solicitação compatível retorna a decisão persistida; alterar os atributos de uma referência existente é recusado.

O contrato utiliza moeda BRL, valores em centavos, prazo de expiração e resultados `APPROVED`, `DECLINED` ou `EXPIRED`. A autenticação usa a credencial específica `PAYMENT_PROVIDER_API_KEY`. A separação permite exercitar uma integração HTTP com persistência independente, mantendo o caráter demonstrativo de todo o fluxo financeiro.

Contratos representativos: `/v1/payments` e `/v1/payments/:id`.

Evidência: `services/payment-provider/src/app.js`.

11.7 Notifications: comunicação assíncrona

Tecnologia: Python 3.13, FastAPI, Pika e `psycopg`.

Dados próprios: `notification_db`.

Notifications consome as famílias de eventos `registration.*`, `payment.*` e `event.*`. A mensagem contém os identificadores necessários à composição; dados atuais de usuário e inscrição são obtidos pelas APIs dos serviços responsáveis, sem consultar suas tabelas diretamente.

O registro `deliveries` controla identificador da mensagem, tentativas e estado de entrega. Uma mensagem já marcada como `SENT` não é enviada novamente pelo processamento normal. O consumidor utiliza confirmação manual, repetição limitada e a fila `notifications.failed` para mensagens que atingem o limite de três tentativas.

A entrega pode ocorrer em arquivos locais, no modo `mailbox`, ou por SMTP. Esses efeitos são processados fora da requisição que confirmou a inscrição. Uma falha de comunicação, portanto, não transforma uma confirmação de inscrição em uma transação de e-mail síncrona.

Evidência: `services/notifications/app/consumer.py`; `services/notifications/app/delivery.py`.

11.8 Attendance: presença e credenciamento

Tecnologia: Python 3.13, FastAPI e `psycopg`.

Dados próprios: `attendance_db`.

Attendance é responsável pelo registro de presença. Consulta Registrations para obter a inscrição e Events para validar o evento e a autorização do organizador. Uma presença nova exige inscrição confirmada, evento publicado e atividade dentro da janela de credenciamento, do início até 24 horas após seu término.

O check-in pode ser iniciado manualmente ou por uma credencial QR resolvida pelas APIs. A credencial é vinculada criptograficamente à inscrição, ao usuário e à atividade. A unicidade de `registration_id` no banco impede presenças duplicadas; o registro mantém quem realizou a marcação e quando ela ocorreu.

Contratos representativos: `/attendance/check-ins`, `/attendance/credentials/resolve` e consultas de presença por inscrição ou evento.

Evidência: `services/attendance/app/domain.py`; `services/registrations/app/domain.py`.

11.9 Certificates: emissão e verificação

Tecnologia: Python 3.13, FastAPI, ReportLab e Pillow.

Dados próprios: `certificate_db`.

Certificates obtém pelas APIs a inscrição, o evento e a presença. A emissão de certificado do participante exige inscrição confirmada, presença válida e atividade concluída. A regra é aplicada por atividade, não apenas pelo encerramento do evento.

O serviço persiste um snapshot da emissão, seu resumo SHA-256 e um código de verificação. O PDF e a consulta pública utilizam esses dados, e a validade é reavaliada contra o estado atual das fontes. Há controle de emissão por inscrição, modelos configuráveis pelo organizador, prévia do documento e fluxo específico de certificados para palestrantes.

O resumo de integridade e o código de consulta sustentam a verificação do documento na plataforma. Eles não são apresentados como assinatura digital de uma autoridade certificadora externa.

Contratos representativos: `/certificates`, download, `/certificates/verify/:code`, configurações e prévia por evento.

Evidência: `services/certificates/app/domain.py`; `services/certificates/app/templates.py`; `services/certificates/app/speakers.py`.

11.10 Reporting: projeções e indicadores

Tecnologia: Python 3.13, FastAPI, Pika e `psycopg`.

Dados próprios: `reporting_db`.

Reporting mantém projeções próprias de eventos e inscrições. O consumidor recebe a família `registration.#` na fila `eventhub.reporting`, registra os identificadores processados e consulta as fontes atuais por APIs. A mensagem funciona como gatilho de reconciliação; uma mensagem atrasada não é aplicada cegamente como uma transição de estado definitiva.

As consultas de relatório também reconciliam dados de Events, Registrations e Attendance, com bloqueio local por evento. Informações financeiras são consultadas em Payments. O serviço calcula inscrições por estado, ocupação, vagas disponíveis, presenças, série de inscrições por dia e valores simulados, além de exportar CSV.

Uma fonte financeira indisponível é indicada explicitamente no resultado. A separação entre projeção e fontes operacionais mantém a propriedade dos dados e torna visível a consistência eventual da integração.

Contratos representativos: `/reports/events/:id` e `/reports/events/:id/export.csv`.

Evidência: `services/reporting/app/domain.py`; `services/reporting/app/consumer.py`.

12. Funcionalidades implementadas

Jornada	Comportamento disponível	Domínios envolvidos
Acesso e perfil	Cadastro, autenticação, recuperação, perfil e vínculo institucional.	Identity.
Descoberta e participação	Catálogo, detalhes do evento, inscrição por atividade, reserva e lista de espera.	Events e Registrations.
Pagamento demonstrativo	Escolha do resultado simulado, consulta e reconciliação da transação.	Payments, Provider e Registrations.
Gestão do organizador	Eventos próprios, programação, publicação, avisos e acompanhamento dos inscritos.	Identity, Events e Registrations.
Credenciamento	Consulta da inscrição, resolução da credencial QR e registro de presença.	Attendance, Registrations e Events.
Certificação	Modelos, prévia, emissão, download e verificação pública.	Certificates e fontes de elegibilidade.
Acompanhamento	Dashboard, indicadores por evento e exportação CSV.	Reporting e fontes operacionais.
Comunicação	Mensagens decorrentes de inscrição, pagamento e avisos públicos do evento.	Notifications e outbox de Registrations.

A interface conecta essas capacidades em jornadas contínuas. A disponibilidade de uma ação depende do perfil, da propriedade do recurso e do estado atual da atividade ou inscrição, conforme as regras mantidas pelos serviços.

Evidência: `frontend/src/App.tsx`; `frontend/src/services/`; `gateway/src/routes/business.js`.

13. Regras de negócio e controles

Regra	Tema	Controle implementado
RN01	Gestão do evento	Organizador individual, vínculo institucional compatível e `ownerUserId` do evento.
RN02	Capacidade e duplicidade	Inventário por atividade, bloqueios locais e restrição de inscrição ativa duplicada.
RN03	Reserva paga	Prazo de pagamento configurável, com expiração e liberação da capacidade.
RN04	Lista de espera	Promoção em ordem de criação, com identificador como desempate; nova reserva ou confirmação conforme o preço.
RN05	Confirmação financeira	Aprovação simulada confirma somente uma reserva ainda válida.
RN06	Presença	Inscrição confirmada, evento publicado e janela de check-in válida.
RN07	Certificação	Inscrição confirmada, presença válida e término da atividade; emissão por inscrição.

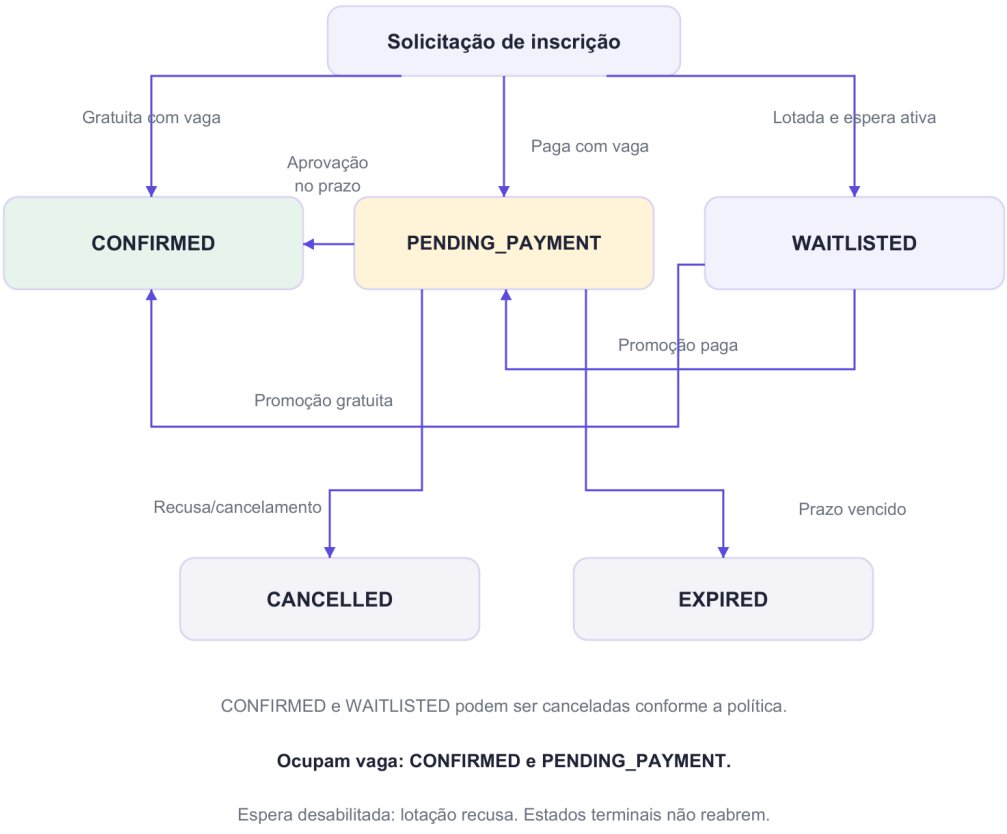


Figura 3: Estados e transições da inscrição por atividade, incluindo reserva paga, espera, confirmação, cancelamento e expiração.

13.1 Invariantes locais e efeitos distribuídos

As transações SQL mantêm as regras que pertencem a cada serviço. Em Registrations, bloqueios e restrições preservam inventário, duplicidade e validade das reservas. Em Payments e no provedor, referências únicas e resultados persistidos permitem repetir chamadas compatíveis. Em Attendance, a presença é única por inscrição; em Certificates, emissão e integridade possuem controles próprios.

A consistência entre serviços é coordenada por contratos e reconciliação, e não por uma transação global. Uma alteração pode estar confirmada em Registrations antes de a notificação ser entregue ou de uma projeção ser atualizada. O sistema mantém os efeitos pendentes em registros recuperáveis, sem exigir que todos os componentes concluam ao mesmo tempo.

Situação	Tratamento implementado
Repetição de inscrição	Chave de idempotência, verificação local e controle de duplicidade ativa.
Disputa pela última vaga	Bloqueio do inventário e restrições de integridade no banco de inscrições.
Falha após iniciar pagamento	Transação pendente persistida e reconciliação por worker.
Pagamento após expiração	Registrations consulta a validade atual antes de confirmar a vaga.
Broker temporariamente indisponível	Mensagens permanecem na outbox até publicação confirmada.
Falha na entrega de mensagem	Estado e tentativas persistidos, repetição e fila de falhas.
Mensagem de relatório repetida	Registro de mensagens processadas e consulta das fontes atuais.
Fonte financeira indisponível	Indicador explicitamente indisponível, em vez de um total zero presumido.

Esses controles constituem mecanismos de recuperação implementados. Não se confundem com uma medição de disponibilidade ou desempenho: a composição local compartilha recursos de infraestrutura e mantém dependências síncronas relevantes, especialmente em Identity.

13.2 Autenticação e autorização

A segurança é aplicada em camadas. Identity assina e valida tokens, controla sessões e versões de credenciais. O gateway verifica assinatura e estado atual da sessão. Os serviços de domínio validam o ator e a propriedade do recurso solicitado. Em eventos, por exemplo, a instituição e o responsável são verificados antes de permitir alterações.

As APIs internas usam `x-internal-key`; o provedor possui uma credencial Bearer específica. Essa autenticação não é descrita como mTLS nem como identidade individual de cada serviço. No ambiente local, somente o proxy publica a porta HTTP no host.

As senhas são derivadas com scrypt e as consultas utilizam parâmetros. Recuperação de senha utiliza tokens de uso único e prazo limitado; alterações de credenciais afetam a validade das sessões. A autorização permanece no backend, independentemente das opções exibidas pela interface.

13.3 Observabilidade

Os serviços propagam `requestId` e `correlationId`. O primeiro identifica uma requisição; o segundo permite reunir etapas do mesmo fluxo, inclusive mensagens e reconciliações. O script `scripts/trace-registration.mjs` utiliza a inscrição para localizar sua correlação e filtrar os logs dos serviços envolvidos.

O endpoint `/api/health` verifica a disponibilidade do gateway. Já `/api/status` consulta a prontidão dos oito serviços de domínio e pode retornar estado degradado quando uma dependência não está pronta. Esses sinais auxiliam a operação, mas uma resposta de prontidão não substitui a validação de um fluxo de negócio completo.

Evidência: `gateway/src/routes/health.js`; módulos de observabilidade dos serviços; `scripts/trace-registration.mjs`.

14. Dados e persistência

O isolamento de dados é lógico: a implantação local reúne nove bancos e seus usuários em uma única instância PostgreSQL. Cada serviço escreve em seu banco e obtém informações externas por contrato HTTP. Identificadores de outro domínio e snapshots são armazenados quando necessários, sem criar chaves estrangeiras entre bancos.

Serviço	Banco	Stack	Dados sob responsabilidade
Identity	`identity_db`	Node/Express	Usuários, instituições, vínculos, sessões e solicitações de recuperação.
Events	`event_db`	Python/FastAPI	Eventos, programação, atividades e informações de publicação.
Registrations	`registration_db`	Python/FastAPI	Inventário, inscrições, decisões de pagamento, recibos e outbox.
Payments	`payment_db`	Node/Express	Transações, decisões solicitadas, resultado do provedor e reconciliação.
Payment Provider	`provider_db`	Node/Express	Transações e respostas persistentes da simulação.
Notifications	`notification_db`	Python/FastAPI	Mensagens recebidas, tentativas e estado das entregas.
Attendance	`attendance_db`	Python/FastAPI	Presenças e identificação do responsável pelo check-in.
Certificates	`certificate_db`	Python/FastAPI	Certificados, snapshots, códigos de verificação e modelos.
Reporting	`reporting_db`	Python/FastAPI	Projeções de eventos e inscrições; mensagens processadas.

O compartilhamento da instância PostgreSQL simplifica a execução acadêmica, mantendo fronteiras de propriedade por banco. Ele não equivale a isolamento físico de infraestrutura. A separação em serviços permite alterar uma implementação sem transformar as tabelas de outro domínio em uma dependência direta.

Evidência: `infra/db/01-init.sh`; manifests e Dockerfiles de `services/`; `compose.yaml`.

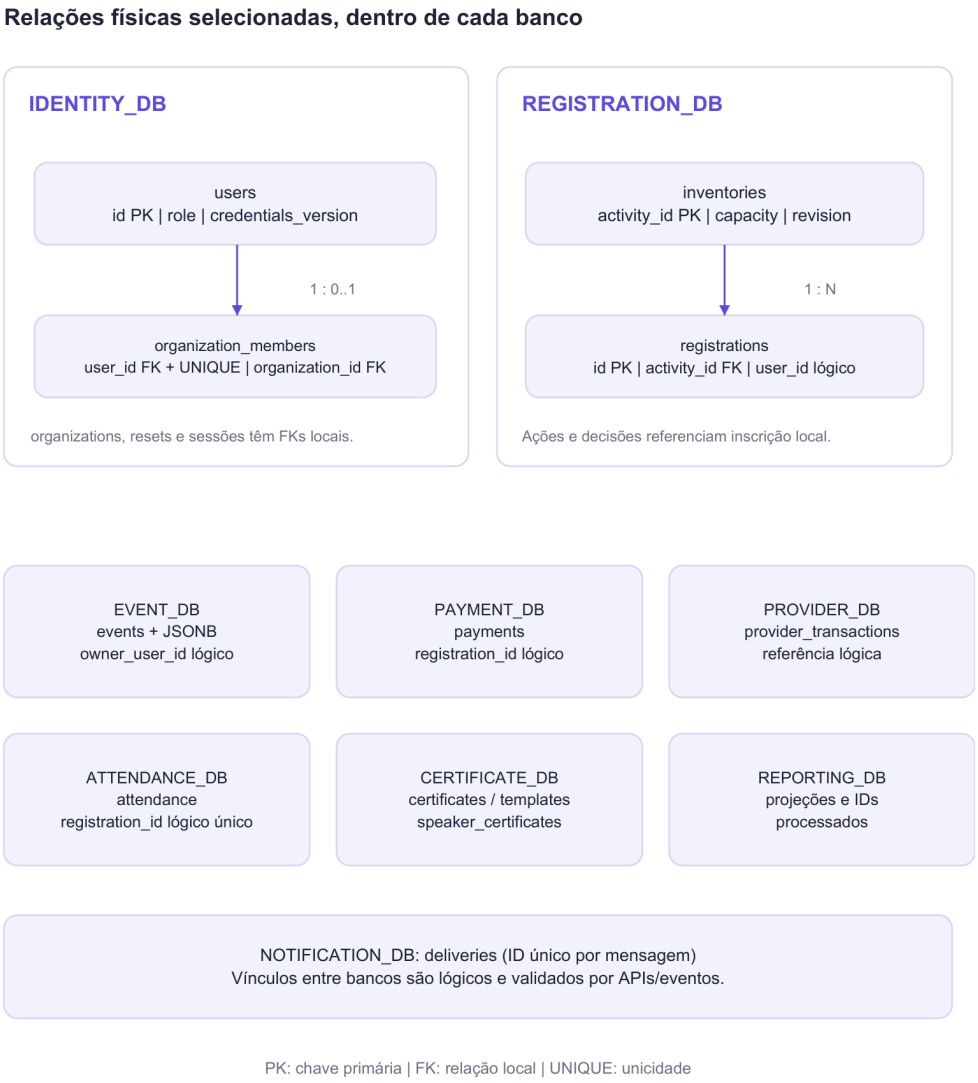


Figura 4: Relacionamentos locais e referências lógicas entre domínios. As relações entre serviços são resolvidas por APIs, sem chaves estrangeiras entre bancos.

15. Fluxos de negócio distribuídos

15.1 Inscrição e ocupação de vagas

O frontend envia evento, atividade e opção de preço com uma chave de idempotência. Registrations consulta as fontes, abre a transação local, serializa a verificação da chave e bloqueia o inventário. Depois de expirar reservas vencidas e considerar a espera, define o resultado e grava a outbox junto com a inscrição.

Estado da inscrição	Significado no fluxo
`CONFIRMED`	Inscrição gratuita ou reserva paga validamente aprovada.
`PENDING_PAYMENT`	Reserva paga com prazo; ocupa uma vaga enquanto válida.
`WAITLISTED`	Aguardando vaga em atividade com espera habilitada.
`CANCELLED`	Cancelamento ou recusa de pagamento tratada pelo domínio.
`EXPIRED`	Reserva cujo prazo ou decisão de expiração encerrou a possibilidade de confirmação.

Cancelamentos e expirações liberam capacidade. Quando aplicável, a próxima inscrição em espera é promovida em ordem FIFO. Se a atividade é paga, a promoção cria uma nova janela de pagamento; se é gratuita, confirma a inscrição. Uma atividade lotada e sem espera habilitada recusa nova inscrição.

15.2 Pagamento simulado e reconciliação

1. Payments consulta a reserva e registra a transação pendente antes de finalizar as chamadas externas.
2. Payment Provider recebe a referência da inscrição como chave idempotente e persiste o resultado da simulação.
3. Payments verifica a resposta e informa o resultado pelo contrato interno de Registrations.
4. Registrations registra a decisão, bloqueia o inventário e verifica a validade atual da reserva. Aprovação confirma somente uma reserva ainda válida; recusa cancela e expiração encerra a reserva.
5. O resultado reconciliado é mantido em Payments. Eventos correspondentes são gravados na outbox de Registrations e seguem para os consumidores.
6. Se uma etapa externa falhar, a transação pendente permanece recuperável pelo worker de reconciliação.

A operação não utiliza uma transação SQL distribuída. A coordenação depende de transações locais, idempotência, decisões persistidas e repetição controlada. Esses mecanismos tratam também a aprovação que chega depois de o prazo ter terminado: o resultado externo não reabre a vaga por conta própria.



Figura 5: Sequência da inscrição paga, com transações locais, provedor simulado, confirmação em Registrations e propagação assíncrona.

15.3 Presença e certificado por atividade

O organizador solicita o check-in ou resolve a credencial QR. Attendance consulta a inscrição e o evento, verifica a autorização e registra uma presença única. Após o término da atividade, Certificates consulta novamente inscrição, evento e presença, verifica a elegibilidade e persiste o documento emitido.

O certificado reúne os dados da atividade e da participação no momento da emissão, com código de verificação e resumo de integridade. As consultas de validade e download reavaliam as condições atuais, preservando a relação entre o documento e as fontes operacionais.

15.4 Consulta de indicadores

Reporting recebe mudanças por mensagens e também reconcilia as fontes durante a consulta. O dashboard organiza os indicadores por evento e período; não acessa os bancos de domínio diretamente. Ocupação considera inscrições confirmadas e reservas com prazo válido. A receita permanece rotulada como simulada e pode ser distinguida dos valores aprovados nas transações do provedor.

16. Integrações e comunicação

16.1 Contratos síncronos

As operações que exigem decisão imediata utilizam HTTP e JSON. O gateway encaminha as chamadas públicas; serviços consultam outros serviços por contratos internos para verificar identidade, titularidade, status e elegibilidade. Downloads preservam os tipos de conteúdo PDF e CSV.

Origem	Destino	Finalidade
Gateway	Identity	Validar sessão atual, além da assinatura do token.
Events	Identity	Consultar ator e vínculo institucional.
Registrations	Identity e Events	Validar participante, programação e estado do evento.
Payments	Registrations e Provider	Obter reserva, executar a simulação e confirmar o resultado.
Attendance	Identity, Events e Registrations	Autorizar organizador e validar inscrição/credencial.
Certificates	Identity, Events, Registrations e Attendance	Autorizar emissão e verificar os requisitos do certificado.
Reporting	Events, Registrations, Attendance e Payments	Reconciliar projeções e montar os indicadores.
Notifications	Identity e Registrations	Compor a mensagem com dados das fontes responsáveis.

As chamadas possuem limites de tempo e traduzem indisponibilidade de dependências em respostas explícitas. A validação no domínio permanece necessária mesmo quando o pedido já passou pelo gateway.

16.2 Mensageria e outbox

A mudança de uma inscrição e a intenção de comunicar seu resultado são persistidas na mesma transação de Registrations. Um worker publica os registros pendentes na exchange durável `eventhub`, do tipo *topic*. A confirmação de publicação e a opção `mandatory` são utilizadas antes de marcar o registro como publicado.

Família	Consumidor	Efeito
`registration.*`	Notifications e Reporting	Comunicar mudança e reconciliar indicadores.
`payment.*`	Notifications	Informar aprovação, recusa ou expiração simulada.
`event.*`	Notifications	Comunicar atualização pública ou cancelamento aos inscritos afetados.

No fluxo de avisos de evento, Events registra o aviso e o encaminha a Registrations por HTTP. Registrations localiza os destinatários e grava as mensagens na sua outbox. Payments, por sua vez, informa sua decisão a Registrations; os eventos financeiros correspondentes também são gerados por essa fronteira de confirmação. Isso evita atribuir a todos os serviços um publicador RabbitMQ que eles não possuem.

Notifications e Reporting utilizam filas próprias e confirmação manual. Seus registros de mensagens processadas e entregas tratam repetições. O modelo admite entrega pelo menos uma vez: não há promessa de envio de e-mail exatamente uma vez se o SMTP concluir o envio e a gravação local do resultado falhar.

Evidência: `services/registrations/app/domain.py`; `services/events/app/announcements.py`; consumidores de Notifications e Reporting.

17. Ambiente e execução

O arquivo `compose.yaml` reúne as imagens da aplicação e a infraestrutura. Os Dockerfiles das aplicações Node.js usam Node 24; os serviços Python usam Python 3.13. O frontend é compilado em uma etapa de build e servido como arquivos estáticos por Nginx.

O provisionamento cria os bancos e usuários de domínio. As aplicações mantêm migrações e inicialização de estruturas em seus pontos de entrada. Os volumes `postgres_data` e `rabbitmq_data` preservam os dados da instância e do broker na operação local.

17.1 Preparação e comandos de operação

A preparação parte de `.env.example`, copiado para `.env` apenas quando o arquivo local ainda não existe. Os scripts abaixo geram os valores ausentes sem imprimir as credenciais:

```
node scripts/init-backend-config.mjs
node scripts/init-identity-keys.mjs
node scripts/init-internal-key.mjs

docker compose up -d -build
docker compose ps
```

Uma instalação que já possui volume pode utilizar `scripts/provision-backend-databases.mjs` para criar bancos e usuários faltantes, após aplicar a configuração atual do PostgreSQL. O procedimento preserva os objetos existentes e não faz rotação automática das senhas já gravadas no banco.

O frontend em execução no Docker corresponde ao build incluído na imagem. Para incorporar alterações na interface, o serviço deve ser reconstruído. O desenvolvimento com Vite pode ocorrer separadamente, utilizando o proxy local para a API. As instruções completas estão no `README.md` da raiz.

17.2 Modos de comunicação por mensagem

No modo `mailbox`, Identity e Notifications gravam arquivos JSON em `/tmp/eventhub-mail` dentro dos contêineres. No modo `smtp`, utilizam o servidor configurado no ambiente. O diretório de mailbox não possui volume no Compose atual; ele não tem a mesma persistência dos bancos e das filas.

A distinção entre armazenamento de estado, retenção de mensagens no broker e efeito final de entrega é parte da operação da solução. O relatório descreve o suporte SMTP implementado sem atribuir a esta revisão uma campanha de entrega externa.

Evidência: `compose.yaml`; `infra/db/01-init.sh`; `scripts/`; `README.md`.

18. Validação e rastreabilidade

Esta edição foi elaborada a partir dos arquivos presentes no repositório em 12 de novembro de 2026. O inventário foi conferido no Compose e no provisionamento dos bancos, e os comportamentos descritos foram relacionados aos contratos, aos modelos persistidos e aos workers implementados.

A análise confirmou os oito domínios, o provedor separado, os nove bancos lógicos e os 14 componentes da composição. Foram examinadas as regras de inscrição e capacidade, a fronteira de confirmação do pagamento, o processamento da outbox, os consumidores, a presença, a elegibilidade de certificados e a reconciliação dos relatórios.

Na verificação desta atualização, a checagem do frontend terminou com código de saída zero, utilizando o comando:

```
tsc -noEmit -noUncheckedSideEffectImports
```

Essa verificação se refere à tipagem do frontend e aos imports de efeitos; não é apresentada como execução de uma suíte integrada do backend.

Os totais de testes e as referências históricas a ambientes de QA do texto anterior não foram reutilizados como resultados atuais. O relatório utiliza caminhos existentes no checkout e não depende de imagens pendentes de inserção. A validação editorial e a renderização deste documento são distintas dos testes funcionais da aplicação.

19. Uso de inteligência artificial na documentação

A inteligência artificial generativa foi utilizada como ferramenta de apoio à elaboração e à revisão da documentação técnica apresentada neste relatório. O apoio concentrou-se na

organização dos tópicos, na melhoria da clareza da redação, na padronização da terminologia e na estruturação das descrições da arquitetura e dos fluxos do EventHub.

O conteúdo documental tem como referência os arquivos do repositório indicados ao longo do texto. A equipe do projeto é responsável por conferir as sugestões, verificar sua correspondência com a implementação e selecionar o conteúdo da versão final. Dessa forma, a IA desempenha um papel de apoio documental, enquanto a responsabilidade técnica e editorial permanece com a equipe.

20. Estado atual da entrega

O escopo acadêmico atual está concluído. A composição contém os oito serviços de domínio e o provedor de pagamento simulado, com APIs e persistência próprias, além dos componentes de acesso e infraestrutura.

Capacidade entregue	Estado na versão atual
Identidade e instituições	Contas, sessões, perfis e vínculos implementados em Identity.
Eventos e participação	Programação, publicação, inventário, reservas e espera implementados em Events e Registrations.
Pagamento simulado	Coordenação, provedor persistente e reconciliação implementados.
Comunicação assíncrona	Outbox, consumidores, controle de entregas e modos mailbox/SMTP implementados.
Presença	Registro manual ou por credencial, com unicidade e validações de domínio.
Certificados	Elegibilidade por atividade, modelos, PDF e consulta de verificação disponíveis.
Indicadores	Projeções, reconciliação, dashboard e exportação CSV disponíveis.
Execução e documentação	Compose, scripts operacionais, README e relatório técnico atualizados.

A conclusão se refere ao recorte descrito neste documento. A inspeção da implementação, a checagem de tipagem e a revisão do relatório são identificadas separadamente das verificações funcionais que exigem execução completa do ambiente.

21. Limites do escopo atual

As características abaixo delimitam a entrega e orientam a interpretação dos resultados:

- **Pagamento demonstrativo:** o provedor mantém transações simuladas; não há cobrança, liquidação ou movimentação financeira real.
- **Implantação local:** Docker Compose reúne os componentes em um ambiente compartilhado. O relatório não atribui a essa configuração uma disponibilidade ou capacidade de produção medida.

- **Isolamento lógico:** os nove bancos pertencem aos serviços, mas utilizam a mesma instância PostgreSQL. A separação de propriedade não implica isolamento físico.
- **Consistência distribuída:** notificações e projeções podem ser atualizadas após a confirmação operacional; outbox, consumidores e reconciliação tratam efeitos pendentes.
- **Comunicação externa:** SMTP é suportado; o modo mailbox mantém arquivos dentro dos contêineres e não possui volume próprio na composição atual.
- **Verificação de certificado:** código público e SHA-256 sustentam a verificação na plataforma, sem representar assinatura digital de autoridade certificadora externa.

Essas escolhas integram o escopo concluído. Alterações de implantação, integração financeira real e ampliação de requisitos devem ser tratadas como uma nova etapa do projeto.

22. Planejamento e resultado alcançado

A relação entre as diretrizes arquiteturais e o resultado entregue permite avaliar o projeto por capacidades implementadas, sem atribuir percentuais de execução ou cronogramas históricos não documentados.

Diretriz arquitetural	Resultado entregue
Separar responsabilidades de negócio	Oito serviços de domínio, com implementações e regras próprias.
Preservar propriedade dos dados	Nove bancos lógicos para os domínios e o provedor simulado.
Integrar decisões imediatas	Contratos HTTP para autenticação, reserva, pagamento, presença, certificação e relatórios.
Desacoplar a propagação de mudanças	Outbox de Registrations, exchange `eventhub` e consumidores em Notifications e Reporting.
Adotar a atividade como unidade	Capacidade, inscrição, pagamento, presença e certificado relacionados à atividade.
Representar uma fronteira externa	Payment Provider com API e banco próprios; Payments coordena e reconcilia o resultado.
Entregar composição reproduzível	Quatorze componentes definidos no Compose, scripts de configuração e instruções no README.

23. Encerramento

O EventHub conclui o escopo atual com os fluxos de identidade, eventos, inscrições, pagamentos simulados, comunicação, presença, certificados e relatórios implementados e integrados. Os serviços de Attendance, Certificates e Reporting fazem parte da implementação entregue, com APIs e persistência próprias.

A principal contribuição técnica é a coordenação de responsabilidades distribuídas. Registrations mantém a decisão sobre vagas e confirmação; Payments coordena uma fronteira externa simulada; Notifications desacopla a entrega das operações principais; Attendance e Certificates preservam suas próprias regras; Reporting compõe indicadores sem acessar diretamente os

bancos de outros domínios. Identity e Events sustentam as regras de acesso e de propriedade que atravessam esses fluxos.

O projeto demonstra a aplicação conjunta de contratos HTTP, propriedade de dados, transações locais, idempotência, outbox, consumidores e reconciliação. As características da versão acadêmica, como pagamento simulado, instância PostgreSQL compartilhada e execução local por Compose, estão explicitadas como decisões do recorte entregue.

Esta etapa é encerrada com a plataforma e sua arquitetura documentadas. Ampliações de escopo ou mudanças de implantação passam a constituir uma evolução posterior do projeto, e não requisitos abertos desta entrega.

Referências técnicas

O mapa abaixo relaciona as descrições deste relatório às fontes de implementação disponíveis no repositório. Os diagramas e a identidade visual seguem o modelo de relatório fornecido para esta edição.

Área	Fontes no repositório
Inventário e bancos	`compose.yaml`; `infra/db/01-init.sh`.
Entrada e gateway	`infra/nginx/nginx.conf`; `gateway/src/routes/business.js`; `gateway/src/security/sessions.js`.
Identidade	`services/identity/src/users/service.js`; `services/identity/src/security/`; `services/identity/src/mail/`.
Eventos	`services/events/app/domain.py`; `services/events/app/announcements.py`.
Inscrições	`services/registrations/app/domain.py`; `services/registrations/app/migrations/`.
Pagamentos	`services/payments/src/payments.js`; `services/payments/src/provider.js`; `services/payment-provider/src/app.js`.
Notificações	`services/notifications/app/consumer.py`; `services/notifications/app/delivery.py`.
Presença	`services/attendance/app/domain.py`.
Certificados	`services/certificates/app/domain.py`; `services/certificates/app/templates.py`; `services/certificates/app/speakers.py`.
Relatórios	`services/reporting/app/domain.py`; `services/reporting/app/consumer.py`.
Interface e operação	`frontend/src/App.tsx`; `README.md`; `scripts/audit-business-data.mjs`; `scripts/trace-registration.mjs`.